Towards a Dependently Typed Language with Observational Equality and QIITs

Qu Zhuoyuan

Contents

0.a Background 3
0.b Benefits 4
0.c What features do we need to do this? 5
0.d But 6
0.e But 7
0.f A Prover with OTT and QIITs 8
0.g Elimination of QITs 9
0.h Elimination of QITs 10
0.i Elimination of QITs 11
0.j How to check? 12
0.k How to check? 13
0.l How to check? 14
0.m How to check? 15
0.n Nested pattern 16
0.o Status and Future Works 17
0.p Other Related References 18

Background

- Altenkirch. Kaposi. *Type theory in type theory using quotient inductive types*[1]

```
data Cx : Type where
  1   : Cx
  _,_ : (Γ : Cx) → Ty Γ → Cx
  ...
data Sb : Cx → Cx → Type where
  id : (Γ : Cx) → Sb Γ Γ
  ...

data Ty : Cx → Type where
  _[_]   : (A : Ty Γ) (γ : Sb Δ Γ) → Ty Δ
  sub-id : (A : Ty Γ) → A [ id ] ≡ A
  Π      : (A : Ty Γ) (B : Ty (Γ , A)) → Ty Γ
  ...

data Tm : (Γ : Cx) → Ty Γ → Type where
  β : {A : Ty Γ} {B : Ty (Γ , A)} (b : Tm (Γ , A) B) → app (lam b) ≡ b
  ...
```

[1] https://dl.acm.org/doi/10.1145/2837614.2837638

Benefits



- `Cx-Sb-Ty-Tm` is initial algebra of CwF.
- Displayed algebras and models can be defined as functions from `Cx-Sb-Ty-Tm` to other types.
- We can extract algorithms from the proof of certain meta-properties
- Some meta-properties (e.g. consistency) are much easier to prove in this setting.

What features do we need to do this?

- QIITs
- UIP
- function extensionality

Only solution we have right now:

Cubical Agda – Glue types + Postulated (Propositional) UIP

But

- We need “Quotient Interleaved Inductive-Inductive Types” (QIIITs)

```
...
data Sb where
  id : Sb Γ Γ <-----------------------------------┐
                                                  |
data Ty where                                     |
  _[_] : (A : Ty Γ) (γ : Sb Δ Γ) → Ty Δ  <------+----------------------┐
  sub-id : (A : Ty Γ) → A [ id ] ≡ A            |                      |
                          ^^--------------------┘                      |
data Sb where                                                         |
  _,_ : (γ : Sb Δ Γ) {A : Ty Γ} (a : Tm Δ (A [ γ ])) → Sb Δ (Γ , A)   |
...                                                ^---^--------------┘
```

- Eliminator of an QIIIT is a set of functions with interleaved clauses
  - Later clauses can depend on definitional equalities introduced by earlier clauses
- This can be solved by using Szumi Xie’s trick
  - Or we can directly implement a prover supporting QIIITs

But

- Transport hell

`Tm` is indexed by `Ty`, which is indexed by `Cx`, we would meet a very serious transport hell problem when we try to do proofs in this setting.

- Solution: Observational Type Theory (OTT)
  - transport can be computed by observing the types

- Other solution:
  - Make some of quotient equalities hold definitionally

    Kaposi. Pujet *Type Theory in Type Theory using a Strictified Syntax*

A Prover with OTT and QIITs

We want a dependently typed language with

- Observational Equality
- Quotient Interleaved Indexed Inductive-Inductive Types

- We need a syntax for pattern matching on QITs.

Elimination of QITs

In CuTT.

```
data ℤ : Type where
  pos    : ℕ → ℤ
  neg    : ℕ → ℤ
  eqZero : pos 0 ≡ neg 0
  --     ^ (i : I) → {ℤ | i = 0 → pos 0, i = 1 → neg 0}

abs : ℤ → ℕ
abs (pos n)    := n
abs (neg n)    := n
abs (eqZero i) := ?

boundary conditions:
i = 0 ⊢ ? = abs (pos 0)
i = 1 ⊢ ? = abs (neg 0)
```

- But in OTT, the equality is not an extension type, so we cannot use the boundary checking to check the elimination of QITs.

Elimination of QITs

Solution: Quotient Equality as Inductive Proposition

```
data ℤ : Type where
  pos    : ℕ → ℤ
  neg    : ℕ → ℤ

data _~z_ : ℤ → ℤ → Prop where
  eqZero : (pos 0) ~z (neg 0)

a ~{ℤ} b := a ~z b

abs : ℤ → ℕ
abs (pos n)    := n
abs (neg n)    := n
```

and we have

$$\frac{f : A \to B \quad A \text{ is a QIT} \quad f \text{ is an eliminator}}{\mathbf{apd}\ f : x \sim y \to fx \sim fy}$$

Elimination of QITs


```
abs : ℤ → ℕ
abs (pos n)     := n
abs (neg n)     := n
```

$$\frac{f : A \to B \quad A \text{ is a QIT} \quad f \text{ is an eliminator}}{\mathbf{apd}\ f : \quad x \sim y \to fx \sim fy}$$

Type checker found a missing clause for **apd** !

```
apd abs eqZero := ?

? : abs (pos 0) ~ abs (neg 0)
```

How to check?

- Löh. Hinze. *Open Data Types and Open Functions*
  - An Haskell extension proposal (psudo syntax here)

```
open data Expr = Num Int
open eval :: Expr -> Int
eval (Num n) = n
-- Another module can add new constructors and clauses
data Expr += Add Expr Expr
eval (Add e1 e2) = eval e1 + eval e2
```

How to check?

- Suppose we have an open inductive type `Elim` and open functions `run` and `apd`

```
open data Elim : (A B : Type) → Type
open run : Elim A B → A → B
open apd : (f : Elim A B) → x ~ y → run f x ~ run f y
```

When we define `abs`, we actually defined

```
data Elim += (abs : Elim ℤ ℕ)
run abs (pos n) := n
run abs (neg n) := n
apd abs eqZero := refl
```

How to check?

For practical implementation

- Generating missing **apd** clauses while coverage checking,

  e.g.

```
abs (pos n)    := n
abs (neg n)    := n
```

  `abs` is splitting on its argument which is a QIT, so we need to generate the missing **apd** clause for `abs`:

```
apd abs (_: x ~ y) : abs x ~ abs y
```

- User must fill all the missing **apd** clauses before the module ends.

How to check?



This approach is quite smart, for example,

```
data I : Type where
  0 : I
  1 : I
  seg : 0 ~ 1

f : Bool → I → ...
f true  i := ...
f false 0 := f0
f false 1 := f1
```

we only have the following missing **apd** clause for `f`:

```
apd (f false) seg : f0 ~ f1
```

Nested pattern

- Nested pattern is tricky

```
f : Maybe I → ...
f (Just 0) := ...
f (Just 1) := ...
f Nothing := ...

missing apd is
apd f (apd Just (_ : x ~ y)) : f (Just x) ~ f (Just y)
```

Hence `apd` is also a pattern.

```
data _~_ : I → I → Prop where
  seg : 0 ~ 1
  apd : (f : A → B) → x ~ y → f x ~ f y
```

Status and Future Works



- We are implementing a prototype of this idea.
- We want to reason about the soundness of this pattern matching algorithm by translating it to eliminators (as in Goguen et al. *Eliminating Dependent Pattern Matching*).
- We want to use this prover to formalize the meta-theories of various type theories. including consistency, canonicity and normalization using gluing techniques.
- Repo is here:

  https://github.com/KonjacSource/ShiTT2/tree/ott

Other Related References



- QIITs is compatible with OTT

  Kaposi. Xie. *Quotient Inductive-Inductive Types in the Setoid Model*